Quantitative Methods – Human Sciences

Potential outcomes and experimental design

Pablo Geraldo Bastías (pablo.geraldo@sociology.ox.ac.uk)

0.1 How to use mathematical notation

You can insert in-line equations using LaTeX~ notation like this: $y_i = \alpha + \beta_\tau \tau + \epsilon_i$ and they will show like this: $y_i = \alpha + \beta_\tau \tau + \epsilon_i$. If you want equations to be separated from the main body, just wrap them around double dollar signs instead of one, like this: $$y_i = \alpha + \beta_\tau \tau + \epsilon_i$$ for the following result:

$$y_i = \alpha + \beta_\tau \tau + \epsilon_i$$

0.2 Simulate a Data Generating Process (DGP)

To insert R code (that will be rendered directly in this document), use the following syntax:

```
# This is R code
# Code and results will be shown in the compiled document

# Set seed for reproducibility
set.seed(1492)

# Let's made a simple calculation
# assuming the following model
# Y0 ~ Norm(10,2)
# Y1 = Y0 + \tau + epsilon_i
# epsilon ~ Norm(0,1)
# tau = 1.5

n <- 1000 # we will create 1K cases
tau <- 1.5 # a treatment effect that is fixed for everyone
epsilon <- rnorm(n) # some "random noise"
y0 <- rnorm(n, mean = 10, sd = 2) # control PO, from normal distribution
y1 <- y0 + tau + epsilon # treated PO, control + treatment effect + noise

# Now, let's create one possible random assignment vector
```

```
# for a binary treatment with p=0.5
d <- rbinom(n, 1, 0.5)

# Let's create the observed outcome variable, using the switching equation
y <- y1*d+(1-d)*y0
```

0.3 Plots and figures

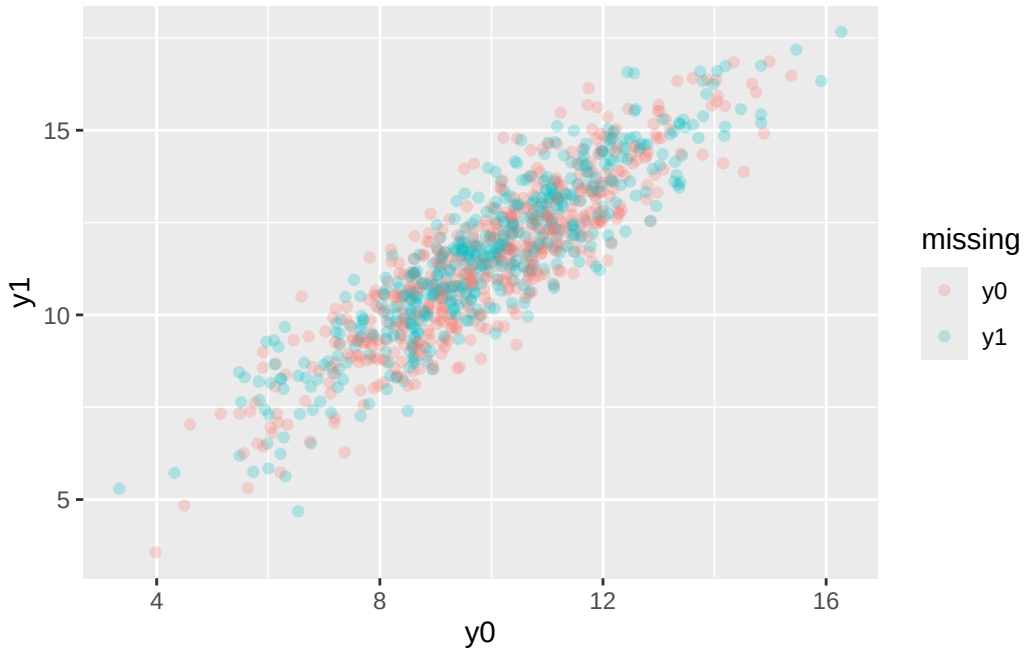
We can plot the data to explore what is in there. Plots will be rendered in the final document if you include their code in this document:

```
# Simple plot example
# First, let's combine our data in a single data.frame

data <- data.frame(y1,y0,y,d)
data$missing <- ifelse(d==1, "y0","y1")

library(ggplot2) # Load ggplot2 package

# Create a scatterplot, highlighting observed vs missing outcomes
ggplot(data, aes(x = y0, y = y1, color = missing)) +
  geom_point(alpha = 0.25)
```



0.4 ATE estimation

```
# Now, let's calculate the ATE
# These two ways are equivalent (why?)

mean(y1-y0)
```

[1] 1.560768

```
mean(y1) - mean(y0)
```

[1] 1.560768

```
# Now, let's approximate the ATE with a difference-in-means
# Think: why cannot we simply use mean(y1-y0 anymore?)
# Think: why does this ATEhat differs from the ATE calculated above?
# Think: is it still a good approximation?

mean(y[d==1]) - mean(y[d==0])
```

[1] 1.439973

1 Now it's your turn

(No worries for now, we will populate this section together in class)

1.1 Randomisation inference

The *sharp null hypothesis* (Fisher) states that the treatment had **no effect for any unit**: $Y_{1i} = Y_{0i}$ for all i. This is a much stronger claim than "the *average* effect is zero". Its great advantage is that, if it is true, we know *both* potential outcomes for every unit: they are both equal to the observed outcome Y_i. Nothing is missing anymore!

That means we can ask: *if the sharp null were true, what difference-in-means would we get under every other assignment that could have happened?* The only source of randomness is the assignment mechanism, which we control, so we can simply re-run it many times while keeping the observed outcomes fixed.

```
# Step 1: calculate the test statistic in the data we actually observed
# (difference in means between treated and control)
obs_diff <- mean(y[d == 1]) - mean(y[d == 0])
obs_diff
```

[1] 1.439973

```
# Step 2: under the sharp null, the outcomes do not depend on d.
# So we keep y fixed and "re-randomise" the treatment many times.
# sample(d) shuffles the treatment vector: same number of treated
# units, but given to different people.

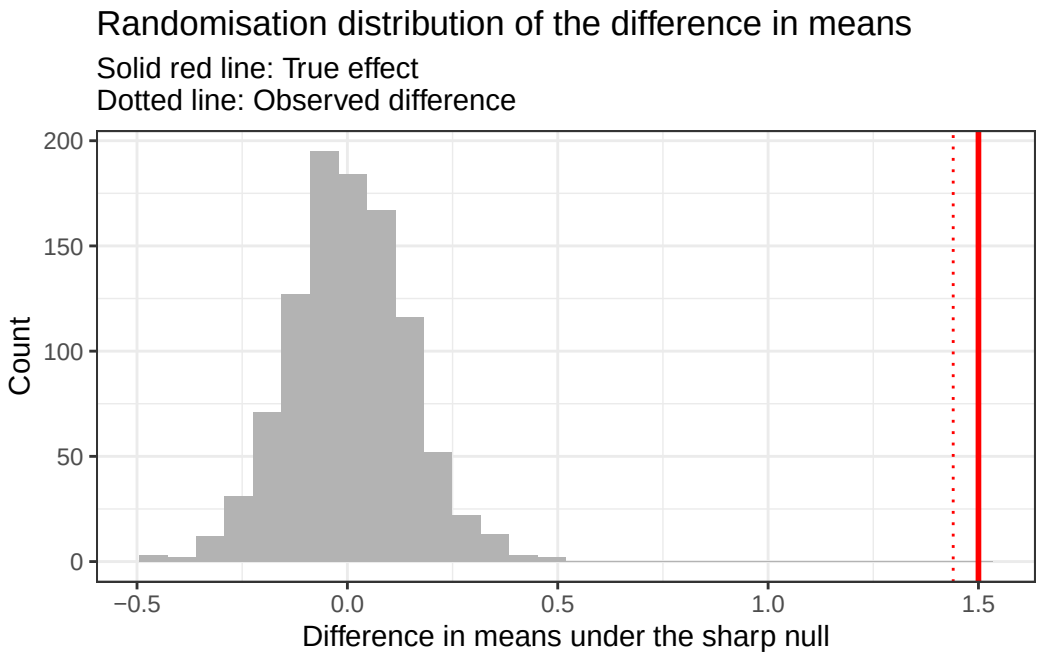
n_perm <- 1000 # number of re-randomisations we are running
perm_diff <- numeric(n_perm) # empty vector to store results (diff-in-means obtained in each randomisation)

for (i in 1:n_perm) {
  d_perm <- sample(d)                          # a new hypothetical assignment
  perm_diff[i] <- mean(y[d_perm == 1]) - mean(y[d_perm == 0]) # same statistic as before
}

# Step 3: the p-value is the share of hypothetical assignments that
# produce a difference at least as extreme (in absolute value) as the observed one
p_value <- mean(abs(perm_diff) >= abs(obs_diff))
p_value
```

[1] 0

```
# Step 4: plot the permutation (randomisation) distribution
# and mark where our observed estimate falls
ggplot(data.frame(perm_diff), aes(x = perm_diff)) +
  geom_histogram(bins = 30, fill = "grey70") +
  geom_vline(xintercept = 1.5, color = "red", linewidth = 1) +
  geom_vline(xintercept = obs_diff, color = "red", linetype = "dotted") +
  labs(x = "Difference in means under the sharp null",
       y = "Count",
       title = "Randomisation distribution of the difference in means",
       subtitle = "Solid red line: True effect \nDotted line: Observed difference") +
  theme_bw()
```



What do we see? The grey histogram shows the differences in means we would get *purely by chance* if the treatment did nothing to anyone. It is centred at zero and almost all values lie between roughly -0.27 and 0.29. Our observed estimate (1.44, red line) is far outside this range: none of the 1000 re-randomisations produced a difference that large, so the p-value is 0 (strictly, $p < 1/1000$). We reject the sharp null of no effect for anyone.

Things to think about: (1) Notice we made **no assumptions** about the distribution of Y (no normality, no large samples): the only thing we used is the assignment mechanism. (2) Why would the p-value be exactly 0 here but not in a smaller experiment? (3) Rejecting the sharp null tells us the treatment did *something* to *someone*. Does it tell us the size of the effect?

To practice: Try modifying some of the simulation parameters and see how your results change!

1.2 Simple randomisation

So far we have looked at a single assignment. But the assignment is random: had the coin flips come out differently, we would have obtained a different estimate. Here we repeat the assignment process 500 times, using the **true potential outcomes** (something we can only do in a simulation!), to see the *sampling distribution* of the difference-in-means estimator.

Under **simple random assignment**, each unit independently flips a coin with $p = 0.5$, as in our DGP above (so the number of treated units varies a little from one replicate to the next).

```
set.seed(1492)

n_sims <- 500
true_ate <- mean(y1 - y0) # the estimand: known only because we simulated it

ate_simple <- numeric(n_sims) # empty vector to store estimates
```

```r
for (i in 1:n_sims) {
  d_sim <- rbinom(n, 1, 0.5)             # 1. a new random assignment
  y_sim <- y1 * d_sim + (1 - d_sim) * y0   # 2. switching equation: what we would observe
  ate_simple[i] <- mean(y_sim[d_sim == 1]) - mean(y_sim[d_sim == 0]) # 3. estimate
}

# Summarise the sampling distribution
true_ate
```

[1] 1.560768

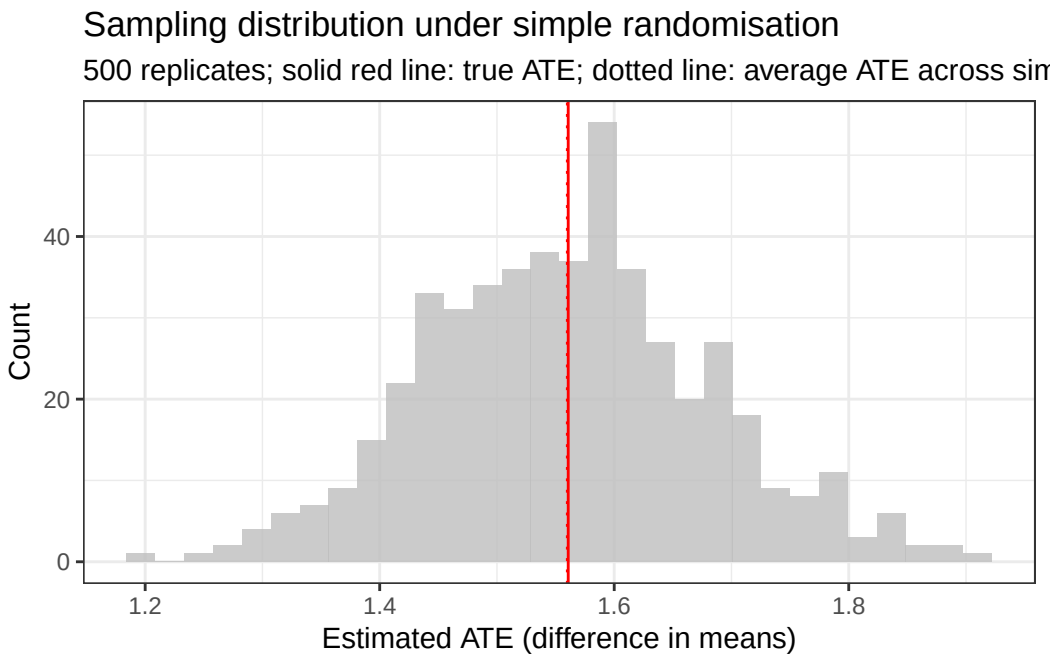
```r
mean(ate_simple) # average of the estimates: is the estimator unbiased?
```

[1] 1.560008

```r
sd(ate_simple)   # standard deviation of the estimates = the standard error
```

[1] 0.1203917

```r
ggplot(data.frame(ate_simple), aes(x = ate_simple)) +
  geom_histogram(bins = 30, fill = "grey", alpha = 0.8) +
  geom_vline(xintercept = true_ate, color = "red") +
  geom_vline(xintercept = mean(ate_simple), color = "red", linetype = "dotted") +
  labs(x = "Estimated ATE (difference in means)",
       y = "Count",
       title = "Sampling distribution under simple randomisation",
       subtitle = "500 replicates; solid red line: true ATE; dotted line: average ATE across sims") + theme_bw()
```



What do we see? Each bar collects estimates from different hypothetical experiments run on the *same* 1,000 people. Individual estimates miss the truth, sometimes by quite a lot, but they are centred on the true ATE: the average estimate is 1.56 vs. a true ATE of 1.561. This is what it means for the difference-in-means to be **unbiased** under random assignment. The spread of the distribution (standard deviation 0.12) is the **standard error**: it tells us how far a single experiment typically lands from the truth.

Things to think about: compare this histogram with the one from randomisation inference. Both come from re-running the assignment, so why is one centred at 0 and the other at 1.5?

1.3 Block randomisation

Now suppose that, *before* the experiment, we observe a binary covariate X that predicts the outcome. We create such a covariate below: it is related to Y_0 but measured with some noise.

In **block (stratified) randomisation** we split the sample into blocks defined by X and run a separate complete randomisation *within each block*, treating exactly half of the units in each one. This guarantees the treated and control groups are perfectly balanced on X in every replicate.

```
set.seed(2025)

# Create a binary covariate that predicts the outcome:
# x = 1 for units with "high" control outcomes (plus some noise,
# so the covariate is predictive but not perfect)
x <- as.numeric(y0 + rnorm(n, sd = 1) > 10)

# Check that it is indeed predictive of the outcome
table(x)
```

```
x
  0   1
503 497
```

```
tapply(y0, x, mean) # mean of Y(0) in each block
```

```
       0        1
 8.595736 11.365753
```

```
ate_block <- numeric(n_sims)

for (i in 1:n_sims) {
  d_sim <- numeric(n) # start with everyone in control

  # within each block, randomly choose half of the units to treat
  for (b in c(0, 1)) {
    units_in_block <- which(x == b)
```

```
  n_treat <- round(length(units_in_block) / 2)
  treated <- sample(units_in_block, n_treat)
  d_sim[treated] <- 1
 }

 y_sim <- y1 * d_sim + (1 - d_sim) * y0 # switching equation

 # estimate the ATE within each block, then average them,
 # weighting each block by its share of the sample
 ate_b0 <- mean(y_sim[d_sim == 1 & x == 0]) - mean(y_sim[d_sim == 0 & x == 0])
 ate_b1 <- mean(y_sim[d_sim == 1 & x == 1]) - mean(y_sim[d_sim == 0 & x == 1])
 ate_block[i] <- ate_b0 * mean(x == 0) + ate_b1 * mean(x == 1)
}

mean(ate_block)
```

```
[1] 1.56185
```

```
sd(ate_block)
```

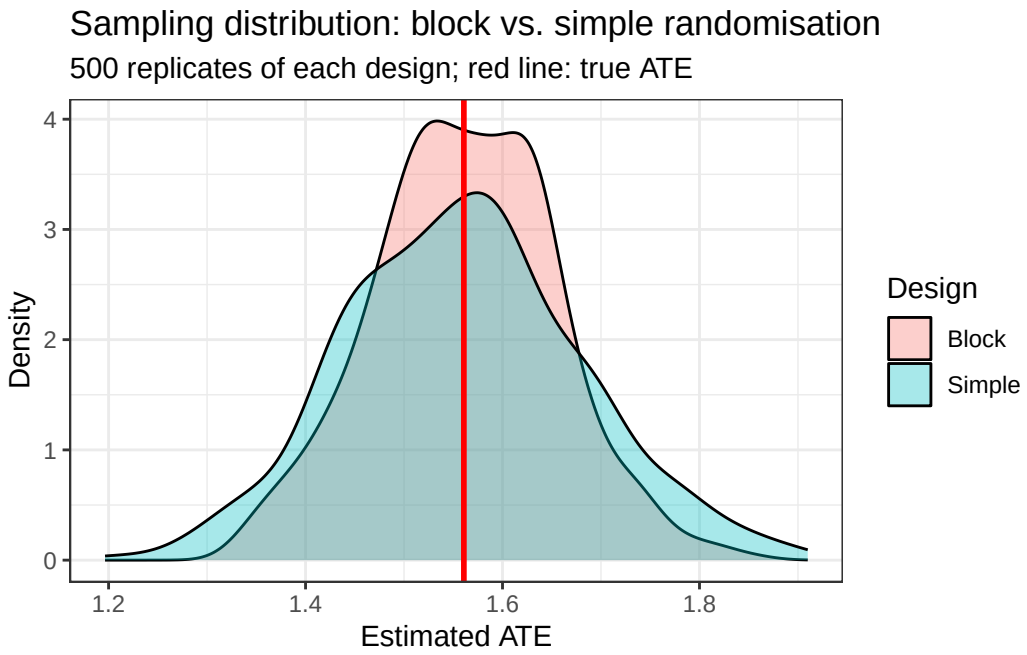
```
[1] 0.09180222
```

```
sd(ate_simple) # compare with simple randomisation
```

```
[1] 0.1203917
```

```
# Plot both sampling distributions together to compare them
sims <- data.frame(
 estimate = c(ate_simple, ate_block),
 design = rep(c("Simple", "Block"), each = n_sims)
)

ggplot(sims, aes(x = estimate, fill = design)) +
 geom_density(alpha = 0.35) +
 geom_vline(xintercept = true_ate, color = "red", linewidth = 1) +
 labs(x = "Estimated ATE",
      y = "Density",
      fill = "Design",
      title = "Sampling distribution: block vs. simple randomisation",
      subtitle = "500 replicates of each design; red line: true ATE") +
 theme_bw()
```



What do we see? Both designs are centred on the true ATE (block average: 1.562), so blocking does not change *what* we estimate. What changes is the **precision**: the standard error falls from 0.12 with simple randomisation to 0.092 with blocking, a reduction of about 24%. Why? Under simple randomisation, some unlucky assignments put more "high-X" units in the treatment group than in the control group, inflating the estimate (or the reverse). Blocking rules out those unlucky assignments, so estimates cluster more tightly around the truth.

Things to think about: what would happen to the gain from blocking if X were *unrelated* to the outcome? Try replacing x with rbinom(n, 1, 0.5) and re-running the chunk.

1.4 Clustered randomisation

Sometimes we cannot assign treatment to individuals, only to **groups**: a new teaching method is given to whole classrooms, a policy to whole villages. In **clustered randomisation**, we randomly select which *groups* are treated, and every unit in a treated group receives the treatment.

Here we split our 1,000 units into 50 groups of 20. Because group membership is assigned at random, it is **uncorrelated with the potential outcomes**: units in the same group are no more similar to each other than units in different groups.

```
set.seed(2025)

# Assign each unit to one of 50 groups (20 units each), at random,
# so group membership is unrelated to the potential outcomes
n_groups <- 50
group <- sample(rep(1:n_groups, each = n / n_groups))
table(group)[1:5] # first few groups: 20 units each
```

```
group
 1  2  3  4  5
20 20 20 20 20
```

```r
ate_cluster <- numeric(n_sims)

for (i in 1:n_sims) {
  # 1. randomly choose half of the GROUPS to be treated
  treated_groups <- sample(1:n_groups, n_groups / 2)

  # 2. a unit is treated if its group was chosen
  d_sim <- as.numeric(group %in% treated_groups)

  # 3. switching equation and difference in means, as before
  y_sim <- y1 * d_sim + (1 - d_sim) * y0
  ate_cluster[i] <- mean(y_sim[d_sim == 1]) - mean(y_sim[d_sim == 0])
}

mean(ate_cluster)
```

```
[1] 1.567443
```

```r
sd(ate_cluster)
```

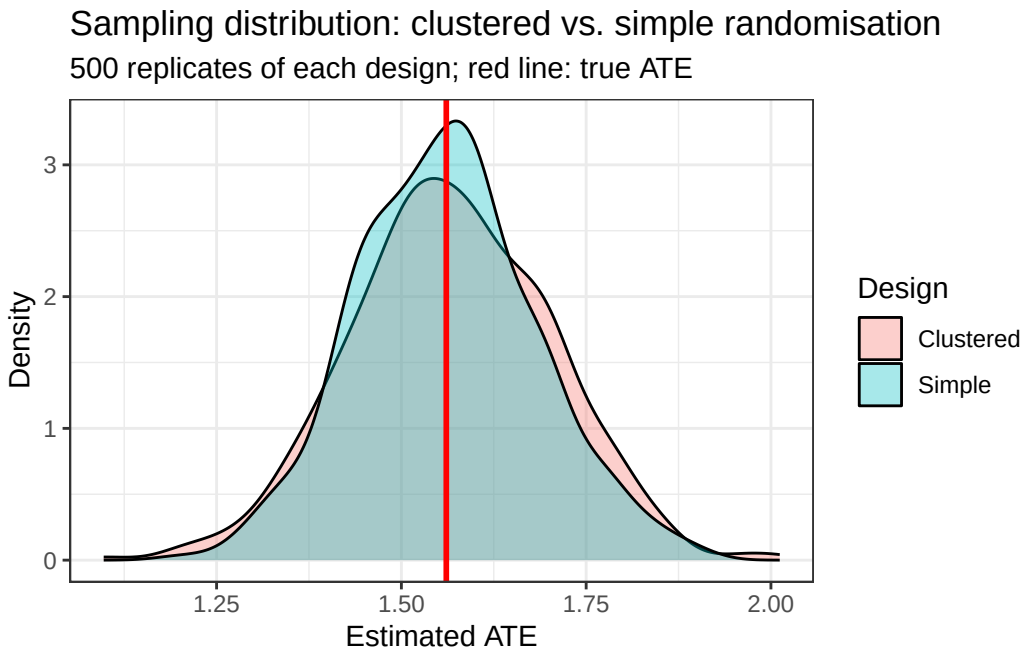
```
[1] 0.1354687
```

```r
sd(ate_simple) # compare with simple randomisation
```

```
[1] 0.1203917
```

```r
sims <- data.frame(
  estimate = c(ate_simple, ate_cluster),
  design = rep(c("Simple", "Clustered"), each = n_sims)
)

ggplot(sims, aes(x = estimate, fill = design)) +
  geom_density(alpha = 0.35) +
  geom_vline(xintercept = true_ate, color = "red", linewidth = 1) +
  labs(x = "Estimated ATE",
       y = "Density",
       fill = "Design",
       title = "Sampling distribution: clustered vs. simple randomisation",
       subtitle = "500 replicates of each design; red line: true ATE") +
  theme_bw()
```



What do we see? The clustered design is also centred on the true ATE (average estimate 1.567), and its standard error (0.135) is very close to that of simple randomisation (0.12). This is because our groups are *arbitrary*: knowing which group a unit belongs to tells us nothing about its potential outcomes, so treating 25 random groups of 20 is effectively like picking 500 individuals at random.

This is the best-case scenario for clustering. In real applications, units in the same cluster tend to be similar (students in the same school, people in the same village), and then treating whole clusters means we effectively have far fewer independent pieces of information than units. The sampling distribution becomes wider, sometimes dramatically so.

Things to think about: modify the code so that groups are correlated with the potential outcomes. For example, replace the line that creates group with group <- ceiling(rank(y0) / (n / n_groups)), which puts units with similar $Y(0)$ in the same group. What happens to the standard error of the clustered design? Why?